

Lenguajes Orientados a Objetos

Introducción

El paradigma orientado a objetos promueve una forma de programación donde las entidades de un problema se categorizan y agrupan según tengan **atributos y comportamiento en común**.

La computación se piensa como un conjunto de **objetos que interactúan entre sí**, donde cada uno puede realizar ciertas tareas.

Un lenguaje orientado a objetos debe brindar soporte para **tres características claves**:

Tipos de Datos Abstractos

La representación de los objetos del tipo se define como una unidad sintáctica simple, y se brinda una interfaz que esconde la representación del tipo.

Herencia

Mecanismo que permite modelar nuevas abstracciones como extensiones o refinamientos de abstracciones existentes.

Ligadura Dinámica de Código

Un mensaje puede ser resuelto de distintas maneras, dependiendo del objeto receptor del mensaje.

Tipo de Dato Abstracto

Un **tipo de dato abstracto** es un tipo de dato definido por el usuario que satisface dos condiciones:

1. La representación de los objetos del tipo son definidos como **unidades sintácticas simples**.
2. La **representación del tipo se esconde del programa que utiliza estos objetos**, por lo tanto las únicas operaciones posibles son aquellas provistas por la definición del tipo.

Tres conceptos importantes

1. El **agrupamiento** es un mecanismo de abstracción que permite agrupar datos (atributos) y subrutinas (métodos) que modelan el comportamiento de un conjunto de entidades.
2. El **ocultamiento de información** permite esconder los detalles de implementación que resultan irrelevantes para los usuarios que van a interactuar con las instancias del TDA.
3. El **encapsulamiento** es una cuestión de diseño del lenguaje. Una abstracción resulta efectivamente encapsulada solo cuando el lenguaje prohíbe el acceso a la información oculta en la abstracción.

Clases, Objetos y Mensajes

Los tipos de datos abstractos en los lenguajes OO se denominan **clases**. Una clase define los atributos y operaciones que van a caracterizar a los objetos de dicha clase.

Las instancias particulares de una clase se denominan **objetos**. Los objetos tienen un determinado estado, comportamiento e identidad (cada objeto es único).

Persona
String nombre Int edad
presentarse(): String salirACaminar()

:Persona
nombre = "José" edad = 30



:Persona
nombre = "María" edad = 25



Perro
String nombre Int edad String raza
alimentarse() darPaseo()

:Perro
nombre = "Toto" edad = 11 raza = "Beagle"



:Perro
nombre = "Luna" edad = 7 raza = "Collie"



Clases, Objetos y Mensajes

En el paradigma OO, toda computación debería ser llevada a cabo mediante mensajes entre objetos. Cuando un objeto le envía un mensaje a otro, este último ejecutará uno de los métodos que tiene públicamente disponibles.

Los mensajes que un objeto puede recibir corresponden a aquellos métodos definidos como públicos en su clase, o bien en una clase ancestro.

```
Persona jose = new Persona("José", 30)
jose.presentarse()
```

"Hola! Mi nombre es José, y tengo 30 años..."

```
Persona maria = new Persona("María", 25)
maria.presentarse()
```

"Hola! Mi nombre es María, y tengo 25 años..."

jose

:Persona

nombre = "José"
edad = 30



maria

:Persona

nombre = "María"
edad = 25



Herencia

Una de las principales preocupaciones en la producción de software se refiere a la reutilización de código.

Los tipos de datos abstractos, a partir del encapsulamiento y los controles de acceso a la representación interna, pueden ser vistos como un medio para lograr la reutilización de código deseada.

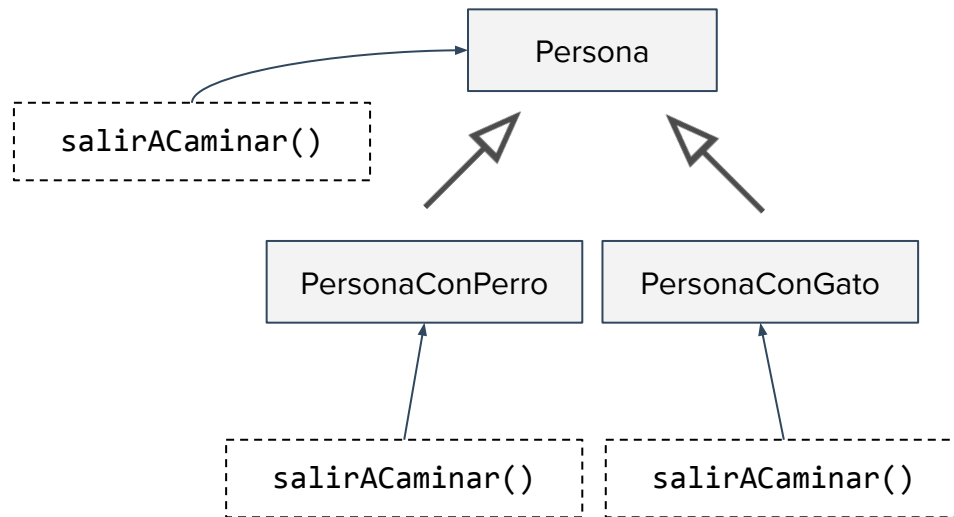
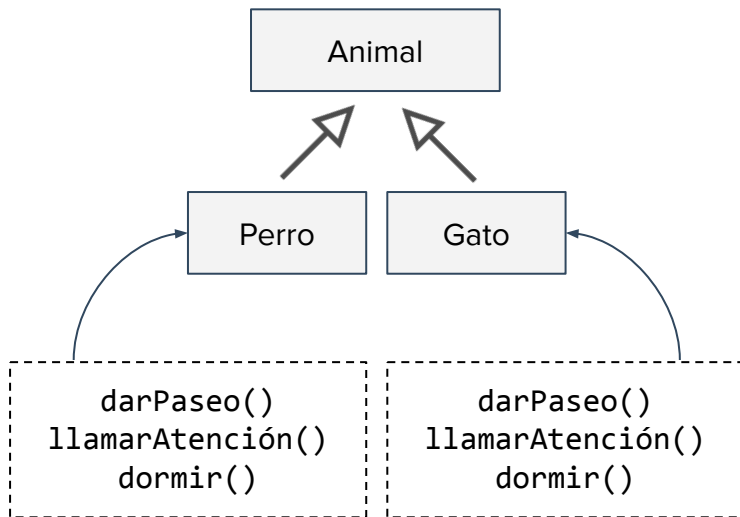
Sin embargo, presentan dos inconvenientes:

- En la mayoría de los casos, las características de un tipo de dato abstracto no son perfectamente adecuadas para un nuevo uso, sino que requieren modificaciones.
- Los tipos de datos abstractos son independientes entre sí, mientras que algunos problemas requieren que existan relaciones entre abstracciones.

El mecanismo de herencia brinda la solución a estos dos problemas.

Herencia

La **herencia** es el mecanismo que permite definir nuevas abstracciones como extensiones o refinamientos de abstracciones existentes.



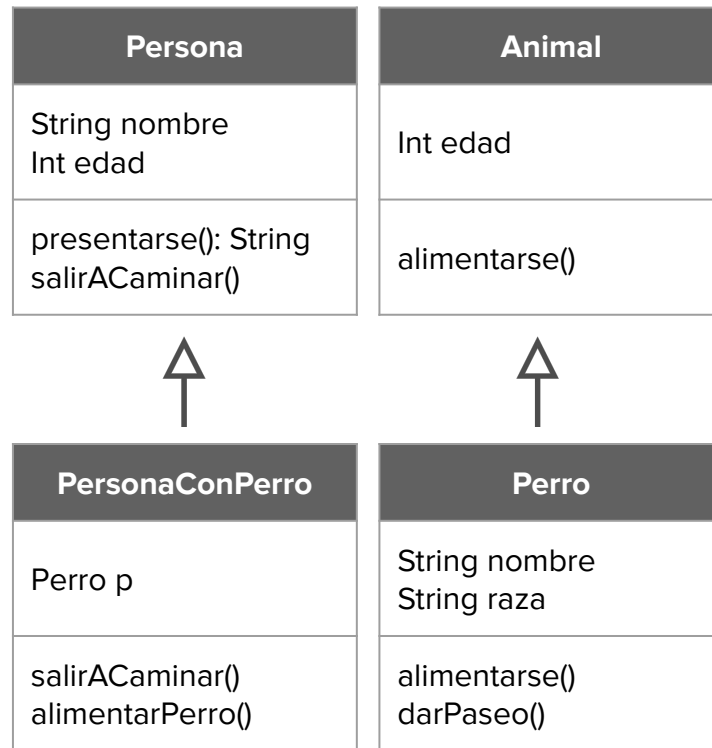
Herencia

Una clase que se define mediante herencia a partir de otra clase se denomina **clase derivada o subclase**.

La clase a partir de la cual se deriva la nueva clase se denomina **clase padre o superclase**.

Una subclase puede diferir de su clase padre en varias formas:

- La clase padre puede tener métodos o atributos privados, no visibles en la clase derivada.
- La clase derivada puede agregar atributos o métodos, que no existen en la clase padre.
- La clase derivada puede modificar el comportamiento de uno o más métodos definidos en la clase padre.



Herencia

Si la relación entre la clase derivada y la clase padre es de tipo **is-a**, se tiene la garantía de que un objeto instancia de la subclase puede aparecer en cualquier lugar en el cual se espera un objeto de la clase padre, sin causar un error de tipos.

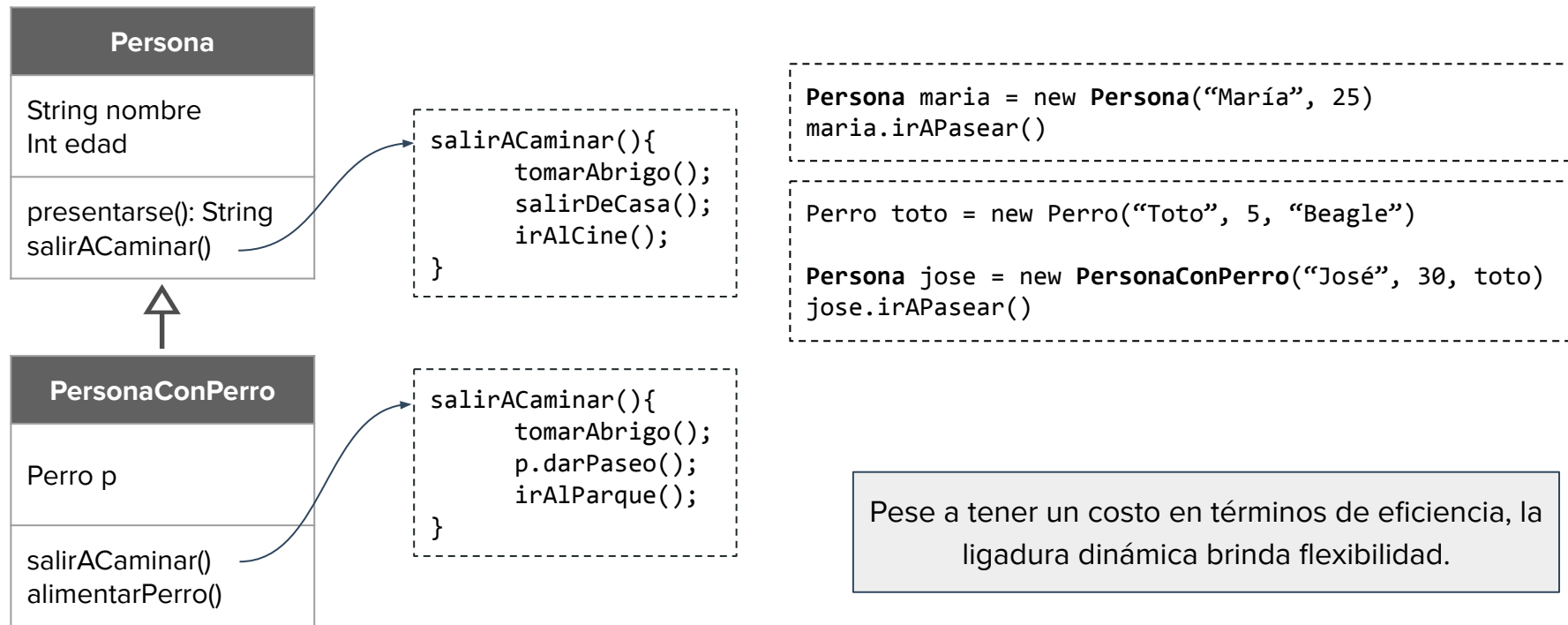
Para que la clase derivada sea subtipo de la superclase, los métodos redefinidos deben tener:

- Mismo número de parámetros que el método redefinido.
- Parámetros de un tipo compatible a los parámetros del método redefinido.
- Retorno de tipo compatible al retorno del método redefinido.



Ligadura Dinámica de Código

La **ligadura dinámica de código** permite que un mensaje se resuelva de distintas maneras dependiendo del tipo del objeto que recibe el mensaje.



Ligaduras Dinámicas vs Ligaduras Estáticas

La **ligadura dinámica** brinda flexibilidad, sobre todo en lenguajes con tipado estático.

Por su parte, la **ligadura estática** resulta más eficiente, por ejemplo, porque el sistema de tipos puede realizar todos los chequeos de tipos antes de la ejecución del programa.

¿Dónde podemos usar ligaduras estáticas en lenguajes orientados a objetos?

- La locación de los atributos y métodos de clase se puede determinar de manera estática (aunque depende de la implementación del lenguaje).
- Los métodos privados tampoco requieren ligadura dinámica (asumiendo que el lenguaje no permite la redefinición en una clase derivada).

Caso de Estudio: Smalltalk

Introducción

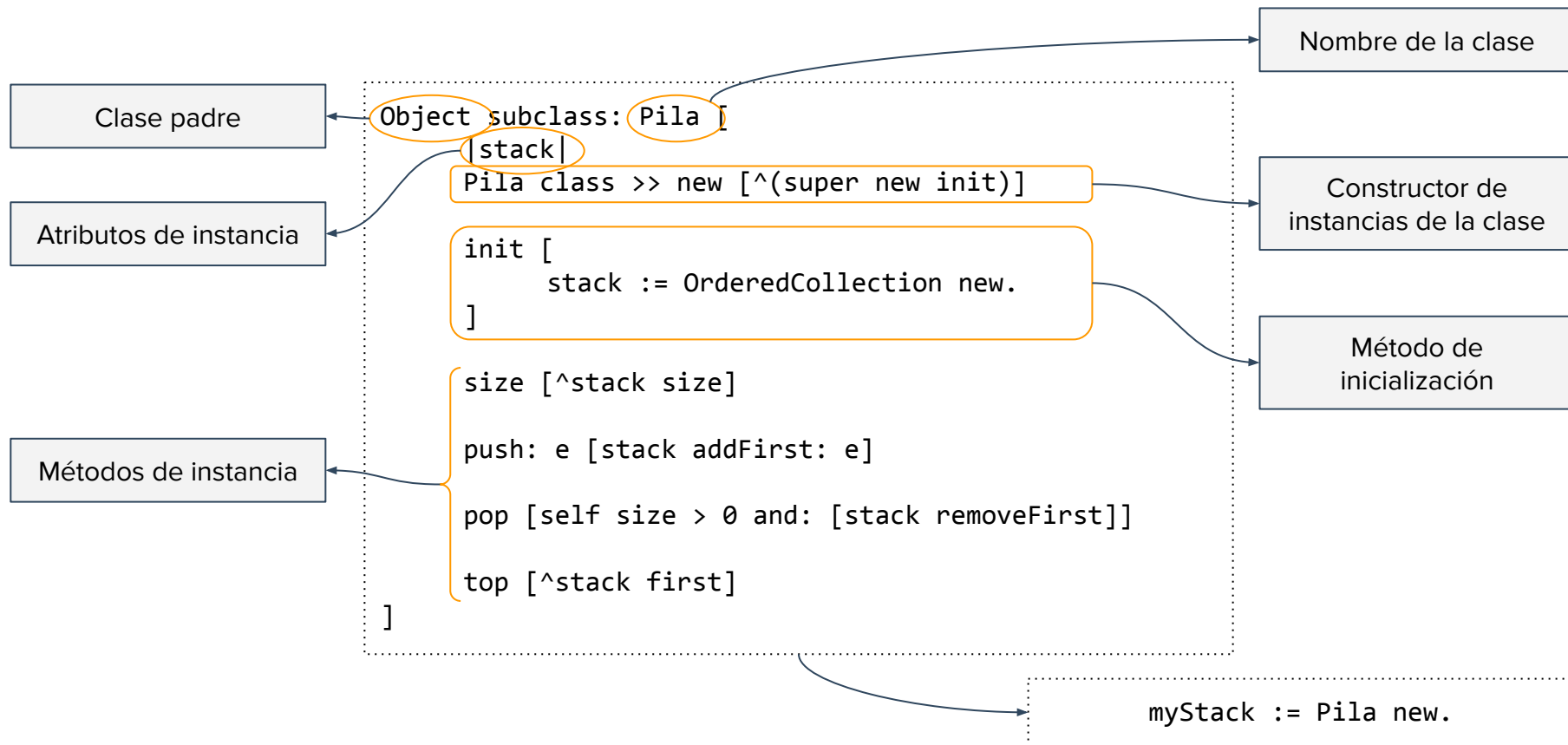
Smalltalk es un lenguaje que soporta el paradigma orientado a objetos de forma pura.

Todo en el lenguaje es un objeto, desde una constante numérica hasta un sistema complejo, y toda computación se realiza a través de mensajes entre objetos.

Es un lenguaje fuertemente tipado, con chequeos de tipos en tiempo de ejecución.

```
Object subclass: Pila [  
    |stack|  
    Pila class >> new [^(super new init)]  
  
    init [  
        stack := OrderedCollection new.  
    ]  
  
    size [^stack size]  
]
```

Definición de una Clase



Definición de Métodos

Para una clase, podemos definir métodos con cualquier número de parámetros siguiendo el esquema:

nombreDelMétodo <: parámetro1 **continuaciónNombre**: parámetro2 ... **continuaciónNombre**: parámetroN>
[... bloque del método ...]

Clase

```
Object subclass: Persona [  
    |nombre edad|  
    ... constructor ...  
    ... métodos de instancia ...  
]
```

Método sin parámetros

```
getNombre [^nombre]
```

Método con un parámetro

```
setNombre: nuevoNombre [  
    nombre := nuevoNombre.  
]
```

Método con dos parámetros

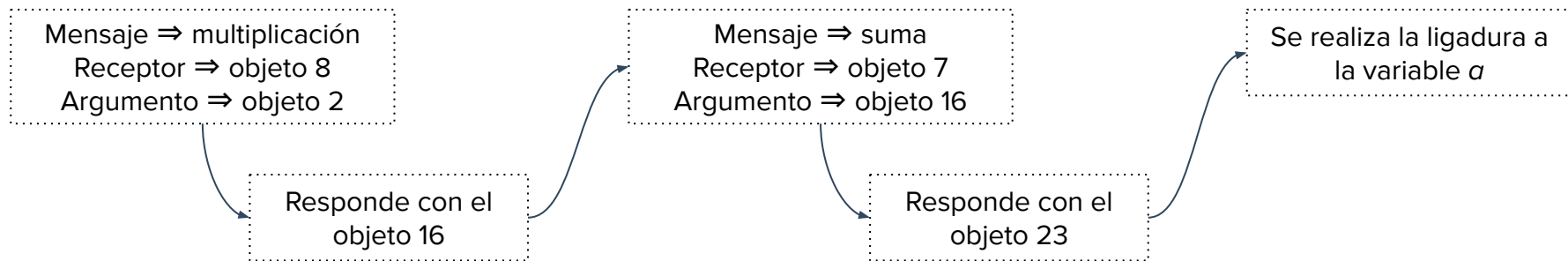
```
setNombre: nuevoNombre yEdad:nuevaEdad [  
    nombre := nuevoNombre.  
    edad   := nuevaEdad.  
]
```

Mensajes

En Smalltalk, **toda entidad en un programa es un objeto**, desde un valor entero hasta un sistema complejo.

Toda computación se realiza por medio de mensajes entre objetos. Un mensaje tiene la siguiente estructura general *<receptor> <cuerpo del mensaje>*.

Por ejemplo, supongamos que tenemos la siguiente asignación: $a := 7 + (8 * 2)$



Todo mensaje tiene un valor de retorno. Si no se devuelve un valor de manera explícita, se retorna una referencia al objeto receptor del mensaje.

Mensajes

El orden en que se evalúan los mensajes en una expresión depende de un esquema de precedencia, que contempla tres clases de mensajes.

Mensajes unarios	<i>Mayor precedencia</i>  <i>Menor precedencia</i>
Sólo interviene el receptor del mensaje	
Ejemplo: <code>myStack top</code>	
Mensajes binarios	
Envían al receptor un mensaje no alfabético e incorporan un argumento	
Ejemplo: <code>7 > 5</code>	
Mensajes con palabra clave	
Envían al receptor un mensaje donde se pueden incorporar múltiples argumentos	
Ejemplo: <code>myStack push: 1</code>	

Mensajes

Consideremos que se desea evaluar la siguiente expresión:

`3 squared + 4 squared between: 1 and: 1000.`

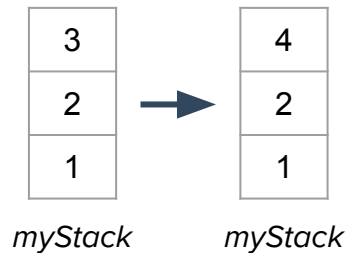
La evaluación sigue los siguientes pasos:

1. Se envía el mensaje `squared` (mensaje unario) al objeto 3.
 - Retorna el objeto 9.
2. Se envía el mensaje `squared` (mensaje unario) al objeto 4.
 - Retorna el objeto 16.
3. Se envía el mensaje de suma (mensaje binario) al objeto 9, con argumento el objeto 16.
 - Retorna el objeto 25.
4. Se envía el mensaje `between: and:` (mensaje con palabra clave) al objeto 25, con argumentos el objeto 1 y el objeto 1000.
 - Retorna el objeto `true`.

Mensajes

Consideremos que se desea evaluar la siguiente expresión:

`myStack pop push: myStack top + 2.`



La evaluación sigue los siguientes pasos:

1. Se envía el mensaje `pop` (mensaje unario) al objeto referenciado por `myStack`.
 - Se desapila el tope de la pila y se retorna una referencia al objeto receptor del mensaje `pop`.
2. Se envía el mensaje `top` (mensaje unario) al objeto referenciado por `myStack`.
 - Retorna el valor almacenado en el tope de la pila.
3. Se envía el mensaje de suma al objeto 2, pasando el objeto 2 como argumento.
 - Retorna el objeto 4.
4. Se envía el mensaje `push` (mensaje con palabra clave) al objeto referenciado por `myStack`.
 - Se apila el objeto 4 y se retorna una referencia al objeto receptor del mensaje `push`.

Mecanismos de Control :: Selección

Las estructuras de selección en Smalltalk se modelan mediante mensajes a objetos.

Clase Boolean

```
<boolean> ifTrue: <bloque> ifFalse: <bloque>
```

Por ejemplo, si queremos verificar si una pila tiene elementos antes de recuperar su tope:

```
e := (myStack empty) ifFalse: [myStack top]
```

```
e := (myStack size > 0) ifTrue: [myStack top]
```

Clase True

```
ifTrue: bloqueTrue ifFalse: bloqueFalse [  
    ^(bloqueTrue value)  
]
```

Clase False

```
ifTrue: bloqueTrue ifFalse: bloqueFalse [  
    ^(bloqueFalse value)  
]
```

Mecanismos de Control :: Iteración

Al igual que con la selección, las estructuras de iteración se modelan mediante mensajes a objetos.

Clase BlockClosure

```
<bloque> whileTrue: <bloque>  
<bloque> whileFalse: <bloque>
```

Clase Integer

```
<entero> timesRepeat: <bloque>
```

Por ejemplo, podemos eliminar todos los elementos de una pila de forma iterativa como sigue:

```
[(myStack empty not)] whileTrue: [myStack pop]
```

```
[(myStack size == 0)] whileFalse: [myStack pop]
```

```
myStack size timesRepeat: [myStack pop]
```

Mecanismos de Control :: Iteración

También podemos utilizar los métodos que provee la clase *Collection*.

Clase Collection

```
<collection> do: <bloque>  
<collection> select: <bloque>  
<collection> reject: <bloque>  
<collection> collect: <bloque>
```

```
col := OrderedCollection new.  
  
col addAll: #(1 2 3 4).  
    col ⇒ OrderedCollection (1 2 3 4)  
  
col do: [:i | i print].  
    ⇒ 1234  
  
pares := col select: [:i | i even].  
    pares ⇒ OrderedCollection (2 4)  
  
impares := col reject: [:i | i even].  
    impares ⇒ OrderedCollection (1 3)  
  
cuadrados := col collect: [:i | i*i].  
    cuadrados ⇒ OrderedCollection (1 4 9 16)
```

Estructuras de Control :: Recursividad

Smalltalk también admite definir métodos recursivos, usando la palabra reservada *self* para hacer referencia al objeto receptor del mensaje.

Por ejemplo, podríamos definir un método fibonacci en la clase Integer de la siguiente forma:

```
Integer extend [  
    fibonacci [  
        ^self < 2      ifTrue: [^self]  
                      ifFalse: [(self - 2) fibonacci + (self - 1) fibonacci].  
    ]  
]  
  
0 fibonacci ⇒ 0  
1 fibonacci ⇒ 1  
10 fibonacci ⇒ 55
```

Notar que estamos agregando comportamiento a una clase predefinida, sin tener que definir una subclase.

Chequeo de Tipos

Smalltalk es un lenguaje **fuertemente tipado**, por lo tanto va a detectar todos los errores de tipos que puedan ocurrir.

Los chequeos de tipos se realizan en tiempo de ejecución, pero no se centran en el tipo de un objeto en sí, sino que se basan en **verificar si los objetos pueden responder a los mensajes que reciben**.

Mientras el objeto receptor de un mensaje pueda responder a ese mensaje, su tipo resulta irrelevante.

<code>myStack new</code>	✓
<code>myStack push: 2</code>	✓
<code>myStack printNl</code>	✓
<code>myStack empty</code>	✗

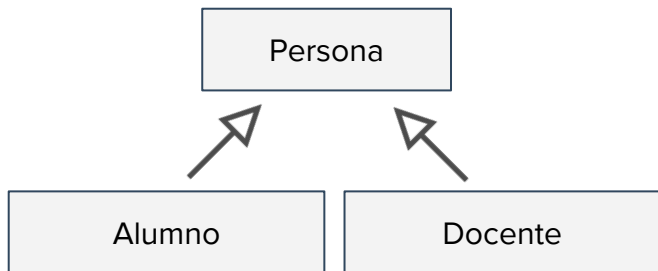
Smalltalk dará un error de tipos si un objeto recibe un mensaje al cual no puede responder.

Herencia

Smalltalk soporta herencia simple. Una subclase en Smalltalk hereda todas las variables de instancia, métodos de instancia y métodos de clase de su clase padre.

Puede agregar nuevas variables y métodos de instancia, y redefinir métodos existentes en su clase padre.

Todas las **subclases son subtipos**, debido a que no es posible esconder atributos y métodos de la superclase.



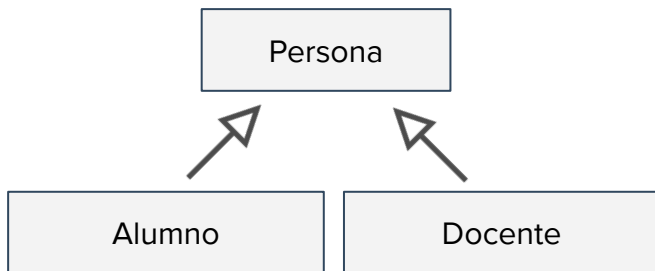
Para ser **subtipo** de su clase padre, la clase derivada debe poder aparecer en cualquier lugar donde se espere un objeto instancia de la clase padre, **sin generar un error de tipos**.

Herencia

```
Object subclass: Persona [  
    |nombre edad|  
  
    Persona class >> new [^(super new init)]  
  
    ... new, init, getters y setters ...  
  
    presentarse [  
        'Hola, mi nombre es ' display.  
        nombre displayNl.  
    ]  
]
```

```
Persona subclass: Docente [  
    |legajo antiguedad materia|  
  
    ... new, init, métodos de instancia ...  
  
    presentarse [... bloque ...]  
]
```

```
Persona subclass: Alumno [  
    |carrera lu promedio|  
  
    ... new, init, métodos de instancia ...  
  
    presentarse: estilo [  
        super presentarse.  
        (estilo = 'formal')  
        ifTrue: [... bloque ...]  
        ifFalse: [... bloque ...]  
    ]  
]
```



¿Preguntas?
